

GIT HOL1

1. Verify master clean state

Command(s):

```
git branch  
git checkout master
```

Expected Output:

```
* master  
Already on 'master'
```

2. Create GitWork branch

Command(s):

```
git checkout -b GitWork  
touch hello.xml
```

Expected Output:

```
Switched to a new branch 'GitWork'
```

3. Update hello.xml

Command(s):

```
echo "Updated content from GitWork branch" > hello.xml  
git status
```

Expected Output:

```
On branch GitWork  
Untracked files:  
hello.xml
```

4. Commit

Command(s):

```
git add hello.xml  
git commit -m "Updated hello.xml in GitWork branch"
```

Expected Output:

```
[GitWork abc123] Updated hello.xml in GitWork branch
```

5. Switch master

Command(s):

```
git checkout master
```

Expected Output:

```
Switched to branch 'master'
```

6. Create hello.xml

Command(s):

```
touch hello.xml  
echo "Hello from master branch" > hello.xml
```

Expected Output:

File created

7. Commit

Command(s):

```
git add hello.xml  
git commit -m "Added hello.xml in master branch"
```

Expected Output:

```
[master def456] Added hello.xml in master branch
```

8. Log

Command(s):

```
git log --oneline --graph --decorate --all
```

Expected Output:

```
* def456 (HEAD->master)  
| * abc123 (GitWork)  
|/  
* initial
```

9. Diff

Command(s):

```
git diff master GitWork
```

Expected Output:

Shows differences in hello.xml

10. Configure P4Merge

Command(s):

```
git config --global diff.tool p4merge  
git config --global merge.tool p4merge
```

Expected Output:

Configuration updated

11. Merge

Command(s):

```
git checkout master  
git merge GitWork
```

Expected Output:

```
CONFLICT (content): Merge conflict in hello.xml
```

12. Status

Command(s):

```
git status
```

Expected Output:

```
Unmerged paths: hello.xml
```

13. Resolve

Command(s):

```
git mergetool
```

Expected Output:

```
Merge tool opened
```

14. Commit merged

Command(s):

```
git add hello.xml  
git commit -m "Resolved merge conflict"
```

Expected Output:

```
[master ghi789] Resolved merge conflict
```

15. Ignore backup

Command(s):

```
git status  
touch .gitignore
```

Expected Output:

```
Created .gitignore
```

16. Commit ignore

Command(s):

```
git add .gitignore  
git commit -m "Added .gitignore"
```

Expected Output:

```
Commit successful
```

17. Branches

Command(s):

```
git branch
```

Expected Output:

```
GitWork  
* master
```

18. Delete

Command(s):

```
git branch -d GitWork
```

Expected Output:

Deleted branch GitWork

19. Final log

Command(s):

```
git log --oneline --graph --decorate
```

Expected Output:

```
* ghi789 (HEAD->master)
```